56. Merge Intervals Medium

Acceptance: 50.9% | LeetCode

Given an array of `intervals` where `intervals[i] = [starti, endi]`, merge all overlapping intervals, and return *an array of the non-overlapping intervals that cover all the intervals in the input*.

Example 1:

```
**Input:** intervals = [[1,3],[2,6],[8,10],[15,18]]
**Output:** [[1,6],[8,10],[15,18]]
**Explanation:** Since intervals [1,3] and [2,6]
overlap, merge them into [1,6].
```

Example 2:

```
**Input:** intervals = [[1,4],[4,5]]
**Output:** [[1,5]]
**Explanation:** Intervals [1,4] and [4,5] are
considered overlapping.
```

Example 3:

```
**Input:** intervals = [[4,7],[1,4]]
**Output:** [[1,7]]
**Explanation:** Intervals [1,4] and [4,7] are
considered overlapping.
```

Constraints:

- `1 <= intervals.length <= 104`
- `intervals[i].length == 2`
- `0 <= starti <= endi <= 104`

Claude Opus Solution

Merge Intervals — Solution Guide

1. Intuition

The **key insight** is: *if we sort intervals by their start time, then any intervals that need to be merged will be adjacent to each other in the sorted order.*

Why? After sorting, every interval's start is ≥ the previous interval's start. So when scanning left to right, an interval can only overlap with the interval(s) we've already processed (specifically, the most recent merged group). This means we never have to "look back" further than the last interval in our result.

Two intervals `[a, b]` and `[c, d]` (with `a ≤ c` after sorting) **overlap** when `c ≤ b` (the second one starts before the first one ends). Note `≤` not `<`, since touching intervals like `[1,4]` and `[4,5]` count as overlapping per the examples.

2. Approach (Pseudocode)

```
1. Sort intervals by start time.
2. Initialize result list with the first interval.
3. For each subsequent interval [start, end]:
     - Look at the last interval in result, call it [last_start, last_end].
     - IF start > last_end:          # gap exists → no overlap
           append [start, end] to result
     - ELSE:                         # overlap → merge
           last_end = max(last_end, end)   # extend the end
4. Return result.
```

3. Python Solution

```python
from typing import List


class Solution:
    def merge(self, intervals: List[List[int]]) -> List[List[int]]:
        # STEP 1: Sort by start time.
        # This guarantees intervals[i][0] <= intervals[i+1][0],
        # so any mergeable intervals become adjacent.
        intervals.sort(key=lambda x: x[0])

        # STEP 2: Seed the result with the first interval.
        # We'll mutate the last entry of `merged` as we go.
        merged = [intervals[0]]

        # STEP 3: Process each remaining interval in sorted order.
```

```
        for current in intervals[1:]:
            last = merged[-1]  # the most recently added (and growable) interval

            # Case A: No overlap — current starts strictly after last ends.
            # Because we sorted, `current` can't overlap anything earlier,
            # so we safely start a new group.
            if current[0] > last[1]:
                merged.append(current)

            # Case B: Overlap (including touching, e.g. [1,4] & [4,5]).
            # Extend the last interval's end. We take max() because
            # `current` might be fully contained, e.g. last=[1,10], current=[2,4].
            else:
                last[1] = max(last[1], current[1])

        return merged
```

---

4. Complexity Analysis

Time Complexity: O(n log n) - Sorting dominates at O(n log n). - The single linear scan afterward is O(n). - Total: O(n log n) + O(n) = **O(n log n)**.

Space Complexity: O(n) (or O(log n) excluding output) - The `merged` output list holds up to n intervals → O(n). - If we don't count the required output, the only extra space is the sort's internal buffer, which is O(n) for Python's Timsort (or O(log n) for in-place sorts). - A common interview answer is **O(log n)** auxiliary space (sorting overhead), since output space is usually not counted.

---

5. Alternative Approaches

Approach	Idea	Complexity
Sorting (this solution)	Sort by start, then sweep & merge adjacent overlaps.	O(n log n) time
Sweep line / Event counting	Treat each start as +1 and each end as -1, sort all events, track active count. Merge while count > 0. Useful when also counting overlaps.	O(n log n) time
Connected components (graph)	Build a graph where overlapping intervals share an edge, then find connected components and merge each. Overkill here but generalizes to harder problems.	$O(n^2)$ time (worse)
Counting sort on values	Since constraints cap values at `10^4`, you could bucket/mark covered ranges in a boolean array of size 10001 and scan it. Avoids the log factor.	O(n + V) time

> **The sorting approach is the standard, optimal choice** for the general case. The counting-sort variant only wins when the value range is small and known (as it is here), but it's less general and rarely expected in interviews.

Pattern Recap

This problem is the canonical **"Sort + Greedy Sweep"** interval pattern. The same template (sort by start, compare with last result) solves related problems like *Insert Interval*, *Interval List Intersections*, and *Meeting Rooms*. Whenever you see overlapping intervals, your first instinct should be: **"Can I sort these to make overlaps adjacent?"**

Codex Solution

1. Intuition

The key pattern is:

> **Sort intervals by their start time, then scan from left to right.**

Once intervals are sorted, any interval that can overlap with the current merged interval must appear immediately after it or later in the sorted order.

For example:

```
[[1,3], [2,6], [8,10], [15,18]]
```

After sorting, we process intervals in increasing start order.

If the next interval starts before or exactly when the current merged interval ends, they overlap:

```
current = [1,3]
next    = [2,6]

2 <= 3, so they overlap
merged = [1,6]
```

Important detail:

```
[1,4] and [4,5]
```

are considered overlapping because the end of the first interval touches the start of the second interval.

So the overlap condition is:

```
next_start <= current_end
```

If they do not overlap, we safely add the next interval as a new separate interval.

---

2. Approach

Pseudocode

```
merge(intervals):

    sort intervals by start time

    result = empty list

    for each interval in intervals:

        if result is empty:
            add interval to result

        else:
            last_interval = last interval in result

            if current interval starts after last_interval ends:
                they do not overlap
                add current interval to result

            else:
                they overlap
                update last_interval end to max(last_interval end, current interval end)

    return result
```

---

3. Python Solution

```python
from typing import List


class Solution:
    def merge(self, intervals: List[List[int]]) -> List[List[int]]:
        # Step 1:
        # Sort intervals by their start time.
        #
        # After sorting, if two intervals overlap, they will appear next to each other
        # or close enough that a single left-to-right scan can merge them.
        intervals.sort(key=lambda interval: interval[0])

        # This list will store the final merged non-overlapping intervals.
        merged = []

        # Step 2:
        # Process each interval in sorted order.
        for start, end in intervals:
            # Case 1:
```

```
            # If merged is empty, there is no interval to compare with.
            #
            # Case 2:
            # If the current interval starts after the end of the last merged interval,
            # then there is no overlap.
            #
            # Example:
            # last merged interval = [1, 3]
            # current interval     = [5, 7]
            #
            # Since 5 > 3, they do not overlap.
            if not merged or start > merged[-1][1]:
                merged.append([start, end])

            else:
                # Case 3:
                # The current interval overlaps with the last merged interval.
                #
                # Example:
                # last merged interval = [1, 4]
                # current interval     = [4, 6]
                #
                # Since 4 <= 4, they overlap.
                #
                # We update the ending point of the last merged interval.
                merged[-1][1] = max(merged[-1][1], end)

        # Step 3:
        # Return all merged intervals.
        return merged
```

---

4. Complexity Analysis

Let `n` be the number of intervals.

Time Complexity: `O(n log n)`

Sorting the intervals takes:

```
O(n log n)
```

The single scan through all intervals takes:

```
O(n)
```

So the total time complexity is:

```
O(n log n)
```

The sorting step dominates.

Space Complexity: `O(n)`

The output list `merged` can contain up to `n` intervals in the worst case.

For example, if no intervals overlap:

```
[[1,2], [3,4], [5,6]]
```

then the output contains all intervals.

Therefore, the space complexity is:

```
O(n)
```

If we ignore the output array, the extra space depends on the sorting implementation. Python's sort may use additional memory internally, but the main algorithmic storage is the result list.

---

5. Alternative Approaches

Alternative 1: Sweep Line

We can treat every interval start and end as events:

```
start = +1
end   = -1
```

Then scan all events in sorted order and track when the number of active intervals goes from `0` to positive, and back to `0`.

This works, but it is more complicated than necessary for this problem.

Alternative 2: Use a Difference Array

Because the constraints say:

```
0 <= starti <= endi <= 10^4
```

we could use an array of size around `10^4` to mark interval coverage.

However, this approach is less general. If interval values were much larger, it would be inefficient.

Alternative 3: Graph Connected Components

We could build a graph where each interval is a node, and two nodes are connected if their intervals overlap. Then each connected component would be merged into one interval.

However, checking all pairs would take:

```
O(n^2)
```

This is much slower than sorting.

Key Insight

Sorting by start time turns the problem into a simple linear scan.

At each step, we only need to compare the current interval with the last merged interval:

```
if current_start <= last_merged_end:
    merge them
else:
    start a new interval
```